ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ

Проект **«Цифровой дизайнер презентаций»**

Архитектура · API · данные · AI-пайплайн · аудит · безопасность · развёртывание · эксплуатация

Содержание

1. Назначение документа
2. Обзор системы
3. Архитектура верхнего уровня
4. Структура репозитория
5. Backend
6. Frontend
7. Модель данных
8. Пайплайн генерации
9. AI-модели
10. Аудит качества
11. Скиллы и версионирование промптов
12. REST API
13. Авторизация и права доступа
14. Безопасность
15. Конфигурация
16. Развёртывание
17. Наблюдаемость и эксплуатация
18. Тестирование
19. Производительность и масштабирование
20. Отказоустойчивость
21. Известные ограничения
22. Устранение неполадок
23. Руководство по развитию
24. Критерии готовности релиза
25. Карта исходников
26. Связанные документы

1. Назначение документа

Документ описывает устройство, развёртывание, эксплуатацию и развитие веб-сервиса «Цифровой дизайнер презентаций». Он предназначен для разработчиков, DevOps-инженеров, тестировщиков, архитекторов и технических экспертов.

Документ дополняет README.md, ARCHITECTURE.md, MODELS.md и AUDIT.md; при расхождении описаний приоритет у кода.

2. Обзор системы

«Цифровой дизайнер презентаций» — веб-сервис, который:

1. принимает корпоративный шаблон PowerPoint;
2. декомпозирует его на дизайн-токены, макеты и повторно используемые паттерны слайдов;
3. анализирует бриф и файлы контент-пакета;
4. формирует план и тексты презентации;
5. создаёт три варианта колоды в стилистике исходного шаблона;
6. добавляет диаграммы, таблицы, инфографику, изображения, анимации и переходы;
7. выполняет автоматический аудит;
8. позволяет совместно редактировать результат;
9. экспортирует презентацию в PPTX, PDF и HTML.

Система работоспособна без подключённых AI-моделей: вместо модельных стадий применяются детерминированные правила и деградационные сценарии. Модели повышают качество структуры, текстов, распознавания паттернов, семантического аудита и иллюстраций.

2.1. Основные пользовательские сценарии

- регистрация, вход и выход из учётной записи;
- просмотр встроенных шаблонов и загрузка собственного шаблона;
- создание презентации по брифу и материалам;
- выбор назначения, аудитории, языка, тона, объёма и оси различий вариантов;
- подключение встроенных и пользовательских скиллов;
- наблюдение за ходом фоновой генерации;
- сравнение трёх вариантов и повторная компоновка выбранного варианта;
- аудит с адресными замечаниями и автоисправлением;
- ручное редактирование слайдов в Студии;
- генерация или пересборка отдельного слайда через AI;
- настройка анимаций, переходов, слоёв и изображений;
- комментарии, присутствие участников и общий доступ по ролям;
- экспорт результата.

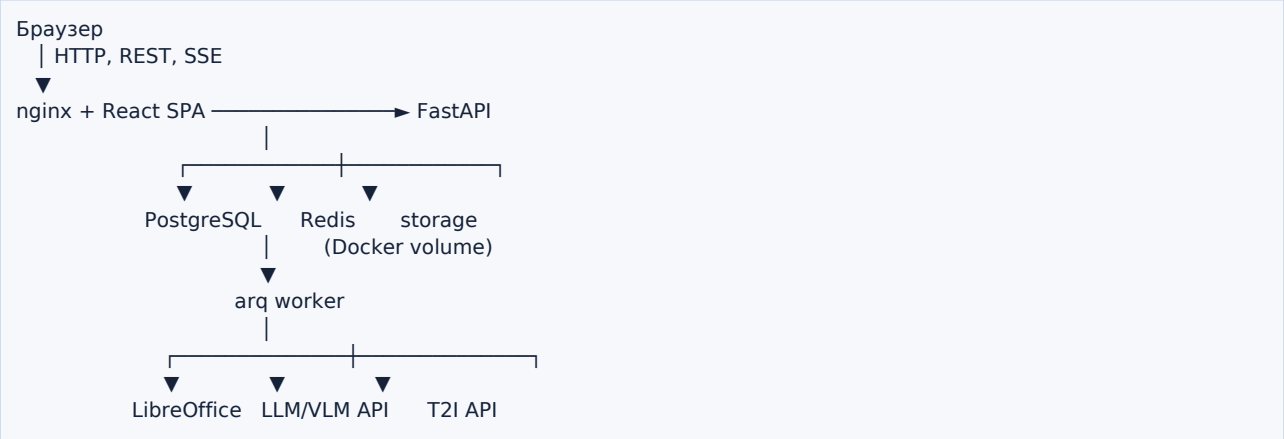
2.2. Термины

Термин	Определение
Шаблон	Исходная презентация, используемая как дизайн-система
Макет	PowerPoint layout с плейсхолдерами и геометрией
Паттерн	Разобранный слайд-пример с семантическими слотами и декором

Термин	Определение
Дизайн-токены	Палитра, шрифты, типографика, сетка, фоны, стили диаграмм и таблиц
План колоды	Нормализованный список спецификаций будущих слайдов
Вариант	Одна из трёх компоновок общего плана
Скилл	Версионируемая инструкция или набор параметров стадии пайплайна
Модель-судья	LLM/VLM, выполняющая контекстуальные проверки аудита
Рендер-факты	Текст, геометрия, стили и изображения, извлечённые из собранного PPTX

3. Архитектура верхнего уровня

Система построена как контейнерное клиент-серверное приложение с отдельным воркером фоновых задач.



3.1. Сервисы

Сервис	Технологии	Ответственность
frontend	React 19, TypeScript 6, Vite 8, TanStack Query, nginx	SPA, маршрутизация, интерфейс редактора; nginx проксирует /api
backend	Python 3.12, FastAPI, SQLAlchemy 2 async, Alembic	REST API, SSE, авторизация, доступ к данным, экспорт и синхронные операции
worker	arq, Python, LibreOffice headless	декомпозиция шаблонов, генерация, рендер, догоняющий аудит
db	PostgreSQL 16	постоянные структурированные данные
redis	Redis 7	сессии, очередь arq, pub/sub, heartbeat генерации
storage	Docker volume	шаблоны, материалы, изображения, PPTX и рендеры

3.2. Ключевые архитектурные принципы

1. **Шаблон трактуется как набор правил.** Слайды-примеры превращаются в паттерны со слотами, а не используются как плоские картинки.
2. **Модель создаёт смысл, код выполняет вёрстку.** Выбор геометрии, построение PPTX и детерминированные проверки не делегируются LLM.
3. **Модельные стадии необязательны.** Отказ или отсутствие провайдера не должны останавливать весь процесс.
4. **Промпты версионируются.** Они хранятся в YAML, а снимок версий сохраняется в каждом прогоне.

5 **Единый контракт слайда.** app.schemas.slide.Slide используется генерацией, вёрсткой, аудитом, экспортом и редактором.

6 **Превью соответствует экспорту.** Превью, PDF и HTML строятся из того же PPTX, который получает пользователь.

4. Структура репозитория

```
.
├── backend/
│   ├── app/
│   │   ├── api/          # маршруты, зависимости, сериализация, роли
│   │   ├── core/         # конфигурация, БД, Redis, безопасность, хранилище
│   │   ├── models/       # SQLAlchemy-модели
│   │   ├── schemas/      # Pydantic-контракты
│   │   ├── pipeline/     # генерация, парсинг, вёрстка, аудит, экспорт
│   │   ├── services/     # прикладные операции над БД и файлами
│   │   ├── main.py       # FastAPI-приложение
│   │   └── worker.py     # конфигурация arq
│   ├── alembic/          # миграции PostgreSQL
│   ├── skills/           # системные и пользовательские YAML-скиллы
│   ├── fonts/            # дополнительные шрифты
│   └── tests/            # автотесты pytest
├── frontend/
│   ├── src/api/          # API-клиент, типы и запросы
│   ├── src/auth/         # провайдер авторизации и guards
│   ├── src/components/   # UI, слайды, редактор и совместная работа
│   ├── src/pages/        # страницы SPA
│   ├── src/lib/          # маршруты и вспомогательная логика
│   └── src/styles/       # токены и стили
├── shablon/              # встроенные PPTX-шаблоны
├── cache/                # готовый кэш декомпозиции встроенных шаблонов
├── docker-compose.yml    # production-подобный запуск
├── docker-compose.dev.yml  # разработка с hot reload
└── .env.example          # пример конфигурации
```

5. Backend

5.1. Запуск приложения

app.main создаёт FastAPI-приложение с базовым префиксом /api. При старте открывается пул arq; при остановке закрываются arq, Redis и SQLAlchemy engine. Интерактивная OpenAPI-документация доступна по /api/docs, схема — по /api/openapi.json.

Перед запуском API контейнер выполняет:

```
alembic upgrade head → python -m app.seed → uvicorn
```

Таким образом, схема БД обновляется автоматически, после чего регистрируются встроенные шаблоны.

5.2. Фоновый воркер

Воркер регистрирует задачи:

Задача	Назначение
parse_template_job	нормализация, разбор и рендер шаблона
run_generation_job	полный генерационный пайплайн
render_variant_job	повторная сборка и рендер варианта
review_variant_job	отложенный модельный аудит

Один воркер принимает до четырёх задач одновременно. Общий timeout задачи — 1500 секунд. Генерация настроена с max_tries=1: автоматический повтор после перезапуска мог бы создать дубликаты вариантов. Живость активной генерации отслеживается heartbeat в Redis.

5.3. Слои backend

- api/routes проверяет запросы, права и формирует HTTP-ответы;
- services реализует прикладные операции и связывает БД с файловой системой;
- pipeline содержит предметную логику преобразования контента;
- schemas задаёт публичные и внутренние контракты;
- models отвечает только за постоянное хранение;
- core инкапсулирует инфраструктурные зависимости.

6. Frontend

Frontend — одностраничное приложение. Серверное состояние кэшируется TanStack Query; доступ к защищённым страницам ограничен guards. Production-сборка раздаётся nginx, маршруты SPA перенаправляются на index.html.

6.1. Маршруты интерфейса

Маршрут	Назначение
/login, /register	вход и регистрация
/	главная панель
/presentations	презентации и проекты
/templates, /templates/:id	библиотека и декомпозиция шаблона
/skills	библиотека скиллов
/presentations/new	новая презентация
/presentations/:id/settings	настройки генерации
/presentations/:id/generation	прогресс пайплайна
/presentations/:id/variants	сравнение вариантов
/presentations/:id/audit	аудит
/presentations/:id/editor	полноэкранная Студия
/presentations/:id/export	экспорт
/share/:token	принятие доступа к презентации
/share/project/:token	принятие доступа к проекту

Переход по /presentations/:id автоматически выбирает страницу по статусу: настройки для черновика, прогресс для активной или неудачной генерации, варианты либо аудит для готовой презентации.

6.2. Обновление данных

- прогресс генерации передаётся через SSE;
- изменения совместной презентации публикуются в Redis pub/sub и приходят клиентам через SSE;
- если поток прогресса разорван, клиент может перейти к периодическому опросу API;
- SSE-события не содержат приватные данные: они служат сигналом повторно запросить авторизованный ресурс.

7. Модель данных

Идентификаторы основных сущностей — UUID. Для JSON-структур используется PostgreSQL JSONB.

Сущность	Основные данные
User	email, имя, Argon2-хеш пароля, активность, стандартные скиллы
Template	владелец, файл, статус, размеры, токены, макеты, роли и паттерны
Project	владелец, имя, цвет и иконка папки
ProjectMember	участник и роль в проекте
ProjectShareLink	capability-токен, роль и активность ссылки
Presentation	владелец, шаблон, проект, бриф, настройки, статус, выбранный вариант
PresentationMember	прямой участник и роль
PresentationShareLink	ссылка доступа к презентации
ContentFile	имя, MIME-тип, размер, путь и извлечённый текст
Generation	статус, прогресс, стадии, снимок скиллов, план и ошибка
Variant	позиция A/B/C, профиль, слайды, состояние рендера и ревизия
AuditIssue	проверка, слайд, адрес элемента, сообщение и состояние
SlideComment	ветка комментариев, координаты, автор, ответ и статус решения
CustomSkill	инструкция, тип, версии и примеры
Notification	тип, текст, ссылка и время прочтения

7.1. Статусы

- шаблон: processing, ready, failed;
- презентация: draft, generating, ready, failed;
- генерация: queued, running, done, failed;
- стадия: pending, running, done, skipped, failed;
- рендер: ready, pending, failed, unavailable.

7.2. Контракт слайда

Slide описывает смысл, содержимое и пользовательские переопределения. Поддерживаемые типы:

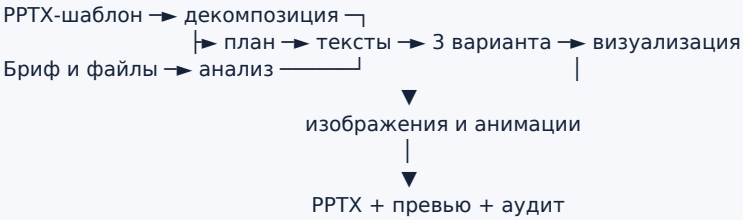
title, section, agenda, bullets, two_columns, cards, chart, table, kpi, timeline, quote, image_text, closing.

Слайд может содержать:

- заголовок, подзаголовок, главный вывод и тезисы;
- колонки или карточки;
- диаграмму, таблицу, KPI, таймлайн или цитату;
- изображение и свободные фигуры;
- заметки докладчика;
- ссылку на паттерн либо макет;
- цвета, шрифты, размеры, выравнивание и рамки элементов;
- crop и радиус изображений;
- скрытые и заблокированные элементы;
- анимации и переход.

Содержимое слайда имеет вычисляемый ключ. Поля анимаций, перехода и блокировки не входят в него, поэтому их изменение не инвалидирует рендер и результаты смыслового аудита.

8. Пайплайн генерации



8.1. Декомпозиция шаблона

Поддерживаются .pptx, .potx, .pptm, .potm, .ppsx, .ppsm; старые форматы .ppt, .pot, .pps, а также .odp и .otp предварительно конвертируются LibreOffice в PPTX.

Парсер извлекает:

- размеры слайда и мастер-тему;
- фактическую палитру и фоновые цвета;
- шрифтовые семейства, размеры и роли текста;
- сетку, поля, логотипы и колонтитулы;
- стили таблиц и цвета рядов диаграмм;
- макеты и их роли;
- слайды-примеры и семантические слоты;
- повторяющийся декор и библиотеку пиктограмм;
- предполагаемый стиль иллюстраций.

OOXML разбирается детерминированно. VLM получает только рендер и уточняет тип, назначение и роли существующих фигур. Её ответ проверяется схемой и порогом уверенности. Ошибка отдельного слайда не прерывает весь разбор.

Для встроенных шаблонов используется cache/; пользовательские шаблоны всегда разбираются заново. Загружаемый OOXML проверяется как ZIP-пакет и защищён от архивов с распакованным объёмом свыше десятикратного лимита загрузки.

8.2. Анализ контент-пакета

Извлечение текста реализовано для:

- TXT, Markdown и CSV;
- PDF;
- DOCX;
- PPTX;
- JSON и YAML;
- PNG, JPEG, WebP и SVG с визуальным анализом или эвристическим описанием.

В промпты передаётся не весь пакет, а ограниченные выдержки: по умолчанию до 4000 символов на файл и до 12000 символов суммарно. Обрезка явно маркируется, чтобы модель не воспринимала выдержку как полный документ.

8.3. Планирование структуры

Планировщик получает название, назначение, бриф, выдержки материалов, аудиторию, язык, тон, желаемый объём, пользовательские скиллы и возможности шаблона. Результат содержит для каждого слайда:

- тип и место в сценарии;
- тему и главный вывод;

- обязательные факты;
- форму подачи;
- данные для визуализации;
- заметки докладчика.

План сохраняется в `Generation.plan`. Повторная компоновка варианта использует этот план и обычно не требует нового обращения к модели.

8.4. Написание и редактура текста

LLM пишет слайды параллельно, по одному запросу на слайд, сразу под ожидаемую структуру. Затем редактор колоды проверяет связность. Ответы валидируются JSON-схемой и нормализуются. Если модель не вернула пригодный JSON, используются факты из плана или тезисы брифа.

Факты и числа считаются защищёнными: редактура и автоисправления не должны вводить значения, отсутствующие в брифе или материалах.

8.5. Компоновка трёх вариантов

Все варианты используют общий смысловой план, но отличаются по выбранной оси:

Ось	Отличие вариантов
density	плотность и объём блоков
visualization	степень визуализации данных
structure	группировка и сценарная структура
layouts	подбор паттернов и макетов

Matcher оценивает соответствие типа, вместимости и наличия необходимых слотов. Сначала выбирается паттерн слайда-примера; при его отсутствии используется макет. Если текст существенно превышает ёмкость слота, отдельная LLM-операция сокращает его без изменения фактов.

8.6. Визуализации

Код преобразует структурированные данные в нативные объекты:

- столбчатые, горизонтальные, линейные, круговые и кольцевые диаграммы;
- таблицы;
- KPI-карточки;
- таймлайны;
- колонки и карточки.

Числа не создаются визуализатором. Он работает только с данными плана и материалов.

8.7. Изображения

Сначала подбираются фотографии из контент-пакета. Если подходящего изображения нет и включён `generate_images`, арт-директор формирует сюжет, а T2I создаёт иллюстрацию в стиле шаблона.

Изображения создаются параллельно и ожидаются ограниченное время. Просроченная или ошибочная генерация не останавливает колоду: слайд переводится в текстовый вариант. По умолчанию на один прогон из трёх вариантов создаётся не более восьми уникальных изображений.

8.8. Анимации и переходы

Режимы: `none`, `subtle`, `lively`. Правила учитывают тип слайда, группируют длинные списки и пропускают скрытые элементы. Пользовательские анимации могут сохраняться при повторном применении правил. Эффекты записываются как нативные PowerPoint-анимации.

8.9. Сборка и рендер

Экспортный слой:

- клонирует подходящие слайды-примеры или создаёт слайды на макетах;
- заменяет содержимое семантических слотов;
- сохраняет редактируемые текст, таблицы, диаграммы, фигуры и изображения;
- применяет анимации и переходы;
- исключает скрытые элементы;
- собирает рендер-факты для аудита;
- вызывает LibreOffice для PDF и растровых превью.

8.10. Параллельность и временные ограничения

- тексты слайдов: `LLM_CONCURRENCY`, по умолчанию 4;
- VLM-разметка: `VLM_CONCURRENCY`, по умолчанию 6;
- T2I: `T2I_CONCURRENCY`, по умолчанию 4;
- модельный аудит: `AUDIT_CONCURRENCY`, по умолчанию 6;
- LibreOffice: `RENDER_CONCURRENCY`, по умолчанию 2;
- задачи воркера: до 4 одновременно.

По замерам проекта (`ARCHITECTURE.md`), колода из десяти слайдов на шаблоне VK Tech с Qwen3.8-27B создавалась за 162–217 секунд без иллюстраций и за 211–249 секунд с иллюстрациями. Это ориентир, а не SLA: результат зависит от провайдера моделей, оборудования, размера шаблона и нагрузки.

9. AI-модели

9.1. Роли

Роль	Рекомендуемая модель	Назначение	Обязательность
LLM	Qwen3.8-27B	план, тексты, сокращение, редактирование	нет
VLM	Qwen3.8-27B или совместимая	разметка паттернов и анализ изображений	нет
Судья	мультимодальная Qwen3.8-27B	контекстуальный аудит текста и рендеров	нет
T2I	FLUX.2 [klein] 4B	иллюстрации	нет

Все текстовые и визуальные модели вызываются через OpenAI-совместимый API. LLM, VLM и судья могут использовать один и тот же endpoint и модель.

9.2. Устойчивый клиент

Клиент последовательно пробует:

1. `json_schema`;
2. `json_object`;
3. запрос без `response_format`;
4. запрос без специфического параметра reasoning.

JSON допускается внутри Markdown-блока или с окружающим пояснением. Ответ другого типа либо оборванный JSON признаётся непригодным и передаётся в fallback вызывающей стадии.

Несколько API-ключей задаются через запятую. Запрос выбирает наименее загруженный ключ; ключи, ответившие 429 или 401–403, временно исключаются из оборота.

9.3. Выбор моделей

Qwen3.8-27B соответствует ограничениям на открытые веса и размер до 35B, поддерживает русский язык, изображения и структурированные ответы. Одна модель может закрыть LLM-, VLM- и audit-роли.

FLUX.2 [klein] 4B выбрана как открытая модель до 20B с лицензией Apache 2.0. По материалам проекта, она быстрее альтернативы Ming-Image-0.1-Design на используемых промптах и лучше соответствует спокойному 3D-стилю шаблонов. Известные ограничения — анатомические дефекты в сложных сценах и псевдотекст на предметах; поэтому арт-директор ограничивает число людей и избегает объектов, на которых ожидается читаемый текст.

9.4. Системные требования для собственного инференса

Для Qwen3.8-27B ориентировочно:

Точность	Размер весов	Рекомендуемая GPU-конфигурация
BF16	около 54 ГБ	1×80 ГБ или 2×48 ГБ
FP8	около 27 ГБ	1×48 ГБ
AWQ/GPTQ 4 bit	около 15 ГБ	1×24 ГБ при ограниченном контексте

Для FLUX.2 [klein] 4B указана потребность около 13 ГБ VRAM. Само приложение GPU не требует, если использует внешний AI API.

10. Аудит качества

Аудит независим от генерации и может повторно запускаться после ручных изменений. Он сочетает четыре метода:

- `data` — проверка структуры `Slide`;
- `render` — анализ фактов фактически собранного PPTX;
- `llm` — оценка текста и логики;
- `vlm` — оценка изображений по рендеру.

Каталог и параметры хранятся в `backend/skills/auditor.yaml`; код проверок — в `pipeline/audit`.

10.1. Группы проверок

Группа	Примеры
Вёрстка	переполнение, пересечения, выход за поля, выравнивание, чрезмерное уменьшение текста
Шаблон	использование макетов, палитры, шрифтов, контраста, логотипов
Плотность	число тезисов, длина тезиса, размер таблицы, число рядов диаграммы
Целостность	заглушки, пустые слайды, подписи диаграмм, дубли, растровый слайд
Содержание	инсайтные заголовки, соответствие выводу, единая мысль, подтверждение фактов, логический поток, релевантность изображения, единый язык

Каталог проекта содержит 34 проверки. У каждой есть идентификатор, категория, серьёзность, метод, описание, возможность исправления, fallback и при необходимости инструкция редактору.

10.2. Состояния и кэширование

Проверка может быть выполнена, ожидать рендера, ожидать судью, завершиться fallback-правилом либо быть недоступной. Вердикты кэшируются по содержимому слайда, поэтому после изменения одного слайда остальные не отправляются судье повторно.

Если судья не успевает за AUDIT_WAIT_SECONDS, основная генерация завершается, а review_variant_job дополняет отчёт позднее.

10.3. Исправления

Простые ошибки исправляются детерминированно. Контекстуальные замечания объединяются по слайду и передаются редактору одним запросом. После изменения применяется тот же компоновщик, что и при первоначальной генерации. Результат повторно рендерится; автоматический перезапуск аудита после произвольной ручной правки не выполняется, а отчёт отмечается устаревшим до команды пользователя.

11. Скиллы и версионирование промптов

Системные скиллы находятся в backend/skills/*.yaml. Файл содержит уникальное имя, версию semver, тип skill/agent, runtime, стадию, модель, описание, промпты и параметры.

Основные скиллы:

- template_parser;
- content_analyzer;
- outline_planner;
- slide_writer;
- deck_editor;
- layout_composer;
- slide_editor;
- visualizer;
- image_style, image_director, image_generator;
- auditor.

Пресеты пользовательских правил находятся в backend/skills/presets/. Пользователь может включать стандартные правила и создавать собственные скиллы. Изменение инструкции создаёт новую минорную версию. Снимок реально использованных версий записывается в прогон генерации и отображается в интерфейсе.

12. REST API

Ниже приведён функциональный индекс API. Полные интерактивные схемы запросов и ответов доступны в Swagger после запуска.

12.1. Метаданные и авторизация

Метод	Путь	Назначение
GET	/api/health	проверка доступности API
GET	/api/meta/options	настройки и варианты для формы генерации
GET	/api/meta/skills	системные скиллы
POST	/api/auth/register	регистрация
POST	/api/auth/login	вход

Метод	Путь	Назначение
POST	/api/auth/demo	вход тестового эксперта
POST	/api/auth/logout	завершение сессии
GET	/api/auth/me	текущий пользователь

12.2. Шаблоны и шрифты

Метод	Путь	Назначение
GET/POST	/api/templates	список или загрузка шаблона
GET/DELETE	/api/templates/{id}	детали или удаление шаблона
GET	/api/templates/{id}/layouts/{name}	рендер макета
GET	/api/templates/{id}/slides/{name}	рендер слайда-примера
GET	/api/templates/{id}/icons/{name}	иконка шаблона
GET	/api/fonts/css	динамический @font-face CSS
GET	/api/fonts/files/{key}	файл шрифта

12.3. Презентации и генерация

Метод	Путь	Назначение
GET/POST	/api/presentations	список или создание
GET/PATCH/DELETE	/api/presentations/{id}	чтение, изменение, удаление
POST	/api/presentations/{id}/files	загрузка материалов
GET/DELETE	/api/presentations/{id}/files/{file_id}	получение или удаление материала
POST	/api/presentations/{id}/generate	запуск генерации
GET	/api/presentations/{id}/generation	состояние последнего прогона
GET	/api/presentations/{id}/generation/events	SSE прогресса
GET	/api/presentations/{id}/events	SSE изменений презентации
GET	/api/presentations/{id}/variants	три варианта
POST	/api/presentations/{id}/select-variant	выбор варианта

12.4. Варианты, аудит и Студия

Метод	Путь	Назначение
GET	/api/variants/{id}	вариант со слайдами и темой
GET	/api/variants/{id}/renders/{revision}/{name}	актуальный рендер слайда
GET	/api/variants/{id}/audit	отчёт аудита
POST	/api/variants/{id}/audit/run	перезапуск аудита
POST	/api/variants/{id}/audit/resolve	исправление замечаний
GET	/api/variants/{id}/export/{fmt}	экспорт; fmt — pptx, pdf или html
POST	/api/variants/{id}/reroll	повторная компоновка
PUT	/api/variants/{id}/slides	пакетное сохранение слайдов

Метод	Путь	Назначение
POST	/api/variants/{id}/animate	автоанимации
POST	/api/variants/{id}/upload-image	изображение редактора
POST	/api/variants/{id}/slides/{slide_id}/regenerate	пересборка слайда
POST	/api/variants/{id}/slides/generate	новый AI-слайд

12.5. Проекты, общий доступ и совместная работа

API поддерживает CRUD проектов, участников и ссылок доступа; аналогичные операции существуют для презентаций. Отдельные endpoints обслуживают уведомления, ветки комментариев, ответы, изменение и удаление комментариев, presence heartbeat и выход из presence.

12.6. Коды ошибок

Код	Типичная причина
400	неверный формат файла или некорректное состояние операции
401	нет действующей сессии
403	недостаточная роль
404	сущность или файл не найден
409	экспорт или операция временно недоступны
413	превышен лимит файла
422	запрос не соответствует Pydantic-схеме
500	ошибка рендера, экспорта или непредвиденная ошибка

13. Авторизация и права доступа

13.1. Сессии

Пароли хешируются Argon2. При входе создаётся криптографически случайный токен; в Redis хранится только его SHA-256. Браузер получает токен в httpOnly-cookie, CLI может использовать `Authorization: Bearer <token>`. TTL по умолчанию — 168 часов. Выход немедленно удаляет запись Redis.

13.2. Роли

Роль	Возможности
owner	все операции, включая удаление
coowner	управление содержимым, участниками и ссылками, кроме удаления объекта
editor	изменение содержимого и генерация без управления доступом
viewer	просмотр и экспорт

Доступ может быть прямым либо унаследованным от проекта. Если действуют несколько ролей, выбирается наиболее высокая.

13.3. Ссылки доступа

Ссылка содержит неугадываемый capability-токен и назначаемую роль. Для каждой роли допускается не более одной активной ссылки на объект. Принять приглашение может только авторизованный пользователь.

14. Безопасность

Реализованные меры:

- Argon2 для паролей;
- хранение только SHA-256 от session token в Redis;
- httpOnly-cookie и опциональный `Secure`;
- серверная проверка прав на каждом защищённом ресурсе;
- нормализация имён загружаемых файлов;
- разрешённые расширения контент-пакета и шаблонов;
- проверка структуры OOXML и ограничение zip bomb;
- ограничение размера загрузки;
- запрет path traversal для генерируемых изображений и рендеров;
- удаление файлов только внутри `STORAGE_DIR`;
- XML-парсер без сетевого доступа и entity resolution;
- приватные cache headers для пользовательских ресурсов;
- SSE без передачи содержимого ресурса;
- rate-limit infrastructure в Redis с fail-open поведением при его недоступности;
- модельные ключи передаются только ожидаемому хосту провайдера.

Рекомендации для production:

- включить HTTPS и `COOKIE_SECURE=true`;
- заменить стандартные учётные данные PostgreSQL;
- не публиковать PostgreSQL и Redis наружу;
- хранить API-ключи в secret manager;
- ограничить CORS и trusted hosts на внешнем reverse proxy;
- настроить резервное копирование БД и тома storage;
- добавить централизованные логи, метрики и оповещения;
- ограничить частоту login, upload, generate и export endpoints;
- выполнять антивирусную проверку пользовательских файлов при работе в недоверенной среде.

15. Конфигурация

Все параметры читаются из окружения; `.env.example` служит шаблоном.

15.1. Базовые параметры

Переменная	По умолчанию	Назначение
APP_PORT	8080	внешний порт приложения
POSTGRES_USER/PASSWORD/DB	app/app/slides	PostgreSQL
DATABASE_URL	compose-значение	async SQLAlchemy DSN
REDIS_URL	redis://redis:6379/0	Redis и arq
SESSION_TTL_HOURS	168	срок сессии
COOKIE_SECURE	false	Secure cookie
MAX_UPLOAD_MB	100	лимит одного файла

Переменная	По умолчанию	Назначение
STORAGE_DIR	/data	постоянное файловое хранилище
BUILTIN_TEMPLATES_DIR	/builtin_templates	встроенные шаблоны
TEMPLATE_CACHE_DIR	/cache	кэш их декомпозиции

15.2. Рендер

Переменная	По умолчанию	Назначение
RENDER_TIMEOUT_SECONDS	420	timeout LibreOffice
RENDER_CONCURRENCY	2	число процессов рендера
SOFFICE_PATH	поиск в PATH	явный путь к LibreOffice

15.3. Модели

Для каждой роли используются *_BASE_URL, *_API_KEY, *_MODEL. Дополнительные параметры:

- LLM: timeout, concurrency, max tokens, temperature, reasoning, extra body и лимиты источников;
- VLM: timeout, max patterns, concurrency, max tokens, reasoning и extra body;
- audit: timeout, max tokens, concurrency и wait time;
- T2I: тип API (chat/images), timeout, wait time, concurrency, max images и seed.

В коде также предусмотрен LLM_PLAN_MAX_TOKENS=4000; при внешней настройке его следует передавать напрямую окружением, даже если он не указан в compose-файле.

16. Развёртывание

16.1. Требования

- Docker Engine и Docker Compose v2;
- минимум около 4 ГБ памяти для демонстрационного запуска;
- рекомендуется 4 CPU и 8 ГБ RAM;
- свободный порт 8080;
- интернет-доступ только если используются внешние модели.

GPU приложению не требуется. LibreOffice потребляет сотни мегабайт на процесс; при дефиците памяти задайте RENDER_CONCURRENCY=1.

16.2. Стандартный запуск

```
cp .env.example .env
docker compose up --build
```

После успешного запуска:

- приложение: http://localhost:8080;
- Swagger: http://localhost:8080/api/docs.

Проверка:

```
docker compose ps
docker compose logs -f backend worker
curl http://localhost:8080/api/health
```

16.3. Режим разработки

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build
```

Доступны:

- Vite: http://localhost:5173;
- API: http://localhost:8000/api/docs;
- PostgreSQL: localhost:5432;
- Redis: localhost:6379.

Frontend можно запустить отдельно:

```
cd frontend
npm install
npm run dev
```

Backend без Docker требует Python 3.12, PostgreSQL, Redis, LibreOffice и системные шрифты.

16.4. Обновление

1. создать резервную копию PostgreSQL и storage;
2. получить новую версию кода;
3. проверить изменения .env.example и compose;
4. выполнить docker compose up -d --build;
5. проверить health endpoint и логи миграций;
6. выполнить smoke-тест создания и экспорта презентации.

Миграции применяются автоматически backend-контейнером. Откат приложения не гарантирует совместимость со схемой БД, поэтому перед обновлением нужна резервная копия.

16.5. Резервное копирование

В резервную копию должны входить:

- PostgreSQL volume pgdata или логический pg_dump;
- volume storage с пользовательскими файлами и артефактами;
- .env либо секреты из внешнего хранилища;
- при необходимости Redis volume для сессий и очереди, хотя Redis не является источником долгосрочных бизнес-данных.

17. Наблюдаемость и эксплуатация

Приложение пишет стандартные Python-логи с временем, уровнем и logger name. Docker healthcheck backend вызывает /api/health; зависимости PostgreSQL и Redis имеют собственные проверки.

Для промышленной эксплуатации рекомендуется собирать:

- длительность и исход каждой стадии;
- глубину очереди arq и возраст старейшей задачи;
- heartbeat активных генераций;
- число ошибок и timeout LibreOffice;
- latency, token usage, 4xx/5xx и rate limits модельных API;
- объём PostgreSQL и storage;
- количество SSE-подключений;
- CPU, RAM и I/O воркера.

Стадии уже сохраняют время начала, окончания и человекочитаемые детали. Это можно использовать как основу для метрик и трассировки.

18. Тестирование

В репозитории находится 151 backend-тест. Они покрывают:

- декомпозицию шаблонов и кэш;
- паттерны, макеты и дизайн-токены;
- генерацию с LLM и fallback без неё;
- различимость трёх вариантов и reroll;
- вместимость текста и типографику;
- диаграммы, изображения и crop;
- T2I, ограничения и обработку ошибок;
- анимации и переходы;
- аудит по данным, рендеру и модели-судье;
- экспорт PPTX/PDF/HTML;
- генерацию отдельного слайда;
- пользовательские скиллы;
- поля совместной работы и Студии.

Запуск backend-тестов:

```
docker compose exec backend pytest
```

или локально:

```
cd backend
pip install -r requirements.txt -r requirements-dev.txt
pytest
```

Проверки frontend:

```
cd frontend
npm ci
npm run typecheck
npm run build
```

Рекомендуемый smoke-тест релиза:

1. зарегистрироваться;
2. открыть встроенный шаблон;
3. создать презентацию с 5–10 слайдами;
4. дождаться трёх вариантов;
5. выбрать вариант и запустить аудит;
6. изменить слайд в Студии;
7. скачать PPTX, PDF и HTML;
8. открыть PPTX в PowerPoint/LibreOffice и проверить редактируемость;
9. создать ссылку viewer и проверить доступ вторым пользователем.

19. Производительность и масштабирование

Основные узкие места:

- latency внешних моделей;
- CPU и память LibreOffice;
- последовательная сборка сложного PPTX;

- размер и число шаблонов;
- параллельное создание изображений.

Вертикальное масштабирование воркера улучшает рендер до определённого предела. Увеличение `RENDER_CONCURRENCY` выше двух, согласно замерам проекта, не даёт заметного выигрыша и повышает расход памяти.

При горизонтальном масштабировании необходимо учитывать:

- все backend/worker экземпляры должны видеть общее файловое хранилище;
- Redis и PostgreSQL должны быть общими;
- глобальная, а не только процессная, координация LibreOffice может потребовать отдельного render service;
- sticky sessions не требуются, поскольку сессии находятся в Redis;
- SSE должен поддерживаться балансировщиком без буферизации;
- лимиты модельных провайдеров должны учитываться глобально.

20. Отказоустойчивость

Отказ	Поведение
LLM отсутствует	план и тексты строятся из тезисов брифа
LLM вернула невалидный JSON	ответ отклоняется, используется fallback
VLM отсутствует	паттерны размечаются эвристиками
T2I недоступна	слайд перестраивается без изображения
Судья не отвечает вовремя	генерация завершается, аудит догоняет в фоне
LibreOffice недоступен	часть рендера/экспорта получает `unavailable` или ошибку
SSE разорван	состояние можно перечитать через REST/опрос
Воркер перезапущен	генерация не дублируется; зависший прогон определяется heartbeat
Один AI-ключ исчерпан	запрос переключается на другой ключ

PostgreSQL остаётся источником истины для бизнес-данных. Redis необходим для входа, очереди и live-функций; его потеря завершает активные сессии и нарушает фоновые операции, но не удаляет презентации из БД и storage.

21. Известные ограничения

- качество презентации зависит от структуры и полноты брифа;
- контент-пакет передаётся модели выдержками, а не целиком;
- рендер PowerPoint и LibreOffice может незначительно различаться;
- редкие или проприетарные шрифты требуют установки в `backend/fonts`;
- сложные эффекты исходного шаблона могут быть интерпретированы не полностью;
- AI не гарантирует фактическую истинность текста, поэтому критические материалы требуют экспертной проверки;
- T2I может создавать артефакты и псевдотекст;
- без мультимодального судьи часть смысловых проверок недоступна либо работает по эвристике;
- активная генерация не возобновляется с середины после аварии воркера;
- HTML — представление на основе рендеров, а не полноценный редактор PowerPoint в браузере;

- текущая compose-конфигурация рассчитана прежде всего на одиночный узел.

22. Устранение неполадок

Генерация не начинается

- проверить статус шаблона: он должен быть `ready`;
- проверить `backend` и `worker` через `docker compose ps`;
- проверить доступность Redis;
- найти ошибку enqueue или миграций в логах backend.

Генерация зависла

- посмотреть текущую стадию и heartbeat;
- проверить логи воркера;
- проверить timeout и quota модельного API;
- проверить память Docker и процессы LibreOffice;
- после признания прогона ошибочным запустить генерацию заново.

Нет превью или PDF

- проверить наличие LibreOffice в backend image;
- проверить `RENDER_TIMEOUT_SECONDS`;
- уменьшить `RENDER_CONCURRENCY` при OOM;
- установить отсутствующие шрифты и пересобрать image;
- проверить свободное место в storage.

Модель возвращает пустой ответ

- увеличить `*_MAX_TOKENS`;
- включить отключение reasoning либо передать `chat_template_kwargs` для vLLM;
- проверить поддержку `response_format` провайдером;
- проверить имя модели и ключ.

Изображения не создаются

- убедиться, что в настройках презентации включена генерация изображений;
- проверить `T2I_BASE_URL`, `T2I_MODEL` и тип `T2I_API`;
- увеличить `T2I_WAIT_SECONDS`, если endpoint медленный;
- проверить лимит `T2I_MAX_IMAGES`.

Шрифты отличаются

- добавить `.ttf/.otf` в `backend/fonts`;
- пересобрать backend и worker images;
- проверить `/api/fonts/css` и fontconfig внутри контейнера;
- заново построить рендер варианта.

23. Руководство по развитию

23.1. Новая стадия пайплайна

1. определить вход и выход стадии;
2. добавить реализацию в подходящий пакет `pipeline`;
3. создать или обновить YAML-скилл;
4. зарегистрировать стадию и прогресс;
5. предусмотреть fallback без модели;
6. сохранить версию скилла в snapshot;
7. добавить unit- и сквозные тесты.

23.2. Новый тип слайда

Потребуются согласованные изменения:

- `SlideKind` и связанные Pydantic-поля;
- planner/writer schemas и нормализация;
- matcher/composer;
- PPTX/HTML export;
- React-компонент визуализации;
- аудит и адреса элементов;
- тесты рендера и экспорта.

23.3. Новая проверка аудита

1. добавить описание в `auditor.yaml`;
2. выбрать метод `data`, `render`, `llm` или `vlm`;
3. реализовать проверку либо расширить контракт судьи;
4. определить severity, fallback и fix;
5. добавить тест на дефект и чистый случай;
6. проверить адресную подсветку элемента.

23.4. Изменение БД

После изменения SQLAlchemy-модели создаётся Alembic-миграция:

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml \
  run --rm backend alembic revision --autogenerate -m "описание"
```

Миграцию необходимо проверить вручную, протестировать на копии данных и убедиться в совместимости rolling update либо остановить приложение на время обновления.

24. Критерии готовности релиза

- миграции применяются на чистой и существующей БД;
- backend-тесты проходят;
- frontend typecheck и production build проходят;
- healthcheck стабилен;
- встроенные шаблоны имеют статус `ready`;
- генерация без моделей завершается fallback-колодой;
- генерация с целевыми моделями укладывается в требуемое время;
- экспортируемые PPTX, PDF и HTML открываются;
- замечания аудита адресуются правильным элементам;

- viewer не может изменять данные, editor не может управлять доступом;
- секреты отсутствуют в образах, репозитории и логах;
- резервная копия и процедура восстановления проверены.

25. Карта исходников

Задача	Основные файлы
конфигурация	backend/app/core/config.py, .env.example, docker-compose.yml
API	backend/app/api/router.py, backend/app/api/routes/
права	backend/app/api/deps.py, backend/app/api/roles.py
безопасность	backend/app/core/security.py, backend/app/core/storage.py
модели БД	backend/app/models/
контракт слайда	backend/app/schemas/slide.py
оркестрация	backend/app/pipeline/orchestrator.py, jobs.py, stages.py
парсер шаблонов	backend/app/pipeline/parsing/
контент	backend/app/pipeline/content/
план и тексты	backend/app/pipeline/planning/
компоновка	backend/app/pipeline/layout/
визуализация	backend/app/pipeline/visuals/
экспорт	backend/app/pipeline/export/
аудит	backend/app/pipeline/audit/, backend/skills/auditor.yaml
модели AI	backend/app/pipeline/ai/
скиллы	backend/skills/
frontend-маршруты	frontend/src/App.tsx
Студия	frontend/src/components/studio/, PresentationEditorPage.tsx
тесты	backend/tests/

26. Связанные документы

- README.md — краткое руководство пользователя и запуск;
- ARCHITECTURE.md — детальное обоснование слоёв и пайплайна;
- MODELS.md — модели, расходы, требования и выбор;
- AUDIT.md — каталог проверок и тестовая стратегия;
- /api/docs — актуальная OpenAPI-документация запущенного экземпляра.